

PSYC 51.17: Models of Language and Communication

Lecture 16: Transformer architecture

Week 5, Lecture 2 - Attention is all you need

PSYC 51.17: Models of Language and Communication

Winter 2026

Winter 2026

1. **The Transformer Revolution:** Why it changed everything
2. **Architecture Overview:** Encoder, Decoder, and components
3. **Self-Attention:** The core mechanism (Q, K, V)
4. **Self-Attention Example:** Understanding pronoun resolution
5. **Multi-Head Attention:** Learning diverse relationships
6. **Three Types of Attention:** Self, Masked, Cross

Goal: Understand the Transformer architecture and self-attention

The transformer revolution

"Attention Is All You Need"

Before Transformers (2017):

- RNNs/LSTMs with attention
- Sequential processing
- Hard to parallelize
- Limited context window
- Slow training

After Transformers:

- Parallel processing
- Scales to GPUs/TPUs
- Long-range dependencies
- Fast & effective

Why get rid of RNNs?

Limitations of Recurrent Architectures:

1. Sequential Bottleneck

- Must process token t before $t+1$
- Cannot parallelize across sequence

Concrete Example:

1	Sentence:	
	"The cat	
	that I saw	
2	yesterday at	4
	the park sat	
	down"	

2. Long Range Dependencies

Why get rid of RNNs?

- 9 Transformer: Direct connection!
- 10 - "sat" attends directly to "cat"
- 11 - No information degradation

Transformer Solution: Every token can attend to every other token directly!

Winter 2026

Transformer architecture overview

Key Components:

- **Multi-Head Self-Attention:** Relate all positions to each other
- **Feed-Forward Networks:** Transform representations
- **Residual Connections & Layer Norm:** Training stability
- **Positional Encoding:** Inject position information

Self-attention: the core mechanism

Key idea: Each word attends to all other words in the sequence

Three learned projections:

- **Query (Q):** What am I looking for?
- **Key (K):** What do I contain?
- **Value (V):** What information do I have?

Computation:

Understanding query, key, value

Analogy: Database lookup or information retrieval

Information Retrieval:

- **Query:** Your search query
- **Key:** Document titles/keywords
- **Value:** Document contents

Process:

1. Compare query to all keys
2. Get similarity scores
3. Weight values by scores
4. Return weighted combination

Self-Attention:

- **Query:** What token i is looking for
- **Key:** What token j offers
- **Value:** Information from token j

Process:

1. Compare Q_i to all K_j
2. Get attention scores
3. Weight all V_j by scores
4. Return new representation for i

Scaled dot-product attention

Step-by-step computation with concrete example:

Input: 3 tokens, embedding dim = 4

```
1 # Input embeddings (3 tokens x 4 dims)
2 X = [[0.1, 0.2, 0.3, 0.4], # "The"
3       [0.5, 0.6, 0.7, 0.8], # "cat"
4       [0.2, 0.3, 0.4, 0.5]] # "sat"
5
6 # Step 1: Compute Q, K, V (using learned weights W_q, W_k, W_v)
7 Q = X @ W_q # [3 x 4]
8 K = X @ W_k # [3 x 4]
9 V = X @ W_v # [3 x 4]
10
11 # Step 2: Compute attention scores
12 scores = Q @ K.T # [3 x 3] - each token vs each token
13
14 # Step 3: Scale by sqrt(d_k) to prevent large values
15 scores = scores / sqrt(4) # divide by 2
16
17 # Step 4: Softmax to get attention weights
```

Scaled dot-product attention

```
18 weights = softmax(scores) # rows sum to 1
19
20 # Step 5: Weighted sum of values
21 output = weights @ V # [3 x 4] - new contextual embeddings
```

Winter 2026

Self-attention example: pronoun resolution

Sentence: "The animal didn't cross the street because it was too tired"

Question: What does "it" refer to?

1 | Attention weights when processing "it".

Self-attention allows the model to:

- Resolve pronouns ("it" → "animal", not "street")
- Understand long-range dependencies (8 tokens apart!)
- Capture syntactic and semantic relationships
- Do this in parallel for all positions!

Visualizing the attention matrix

For sentence: "The cat sat on the mat"

To →	The	cat	sat	on	the	mat
From ↓						
cat	0.1	0.5	0.2	0.1	0.05	0.05
sat	0.05	0.3	0.4	0.15	0.05	0.05

Multi-head attention

Why use multiple attention heads?

Intuition:

- Different heads learn different relationships
- Head 1: Syntactic relationships
- Head 2: Semantic relationships
- Head 3: Positional patterns

Formula:

Concrete Example:

1	Sentence: "The cat sat on	
	the mat"	
2		
3	Head 1 (syntax):	
4	"sat" → "cat" (subject-	
	verb)	
5	"mat" → "the"	13
	(determiner)	
6		
7	Head 2 (semantics):	
	"sat" → "the" (action	

Why multiple heads?

Example: Different heads learn different patterns

Sentence: "The cat sat on the mat"

Head 1: Syntactic Dependencies

- "cat" → "The" (noun-determiner)
- "sat" → "cat" (verb-subject)

Head 3: Local Context

- Each word → neighbors
- Short-range dependencies
- N-gram like patterns

Three types of attention

1. Self-Attention (Encoder)

- Each position attends to all positions in same sequence
- Bidirectional: can see past and future
- Used in: BERT, encoder-only models

2. Masked Self-Attention (Decoder)

Preventing the model from "cheating" during generation

Problem: During training, we have the full target sequence. Without masking, the model could "peek" at future tokens!

Solution: Mask out future positions by setting attention scores to -infinity before softmax.

```
1 | # Example: Generating "The cat sat"
```

Result: Token at position t can only attend to positions $\leq t$

- Maintains autoregressive property
- Enables parallel training while preserving causality

Connecting encoder and decoder in seq2seq models

1 | Encoder Outputs $\rightarrow$ (Keys & Values) $\rightarrow$ Decoder State $\rightarrow$ (Queries) $\rightarrow$

Key Properties:

- **Q** comes from decoder (what decoder needs)
- **K, V** come from encoder (what input provides)
- Allows decoder to "look at" relevant parts of input
- Similar to the original attention mechanism from Lecture 12!

Used in: Machine translation, summarization, any encoder-decoder task

Implementing self-attention in PyTorch

Scaled dot-product attention

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class SelfAttention(nn.Module):
7     def __init__(self, embed_dim):
8         super().__init__()
9         self.embed_dim = embed_dim
10        self.W_q = nn.Linear(embed_dim, embed_dim)
11        self.W_k = nn.Linear(embed_dim, embed_dim)
12        self.W_v = nn.Linear(embed_dim, embed_dim)
```

Implementing self-attention in PyTorch

```
13
14 def forward(self, x, mask=None):
15     Q = self.W_q(x) # Queries: what am I looking for?
16     K = self.W_k(x) # Keys: what do I contain?
17     V = self.W_v(x) # Values: what info do I provide?
18
19     # Attention scores: how similar are Q and K?
20     scores = torch.matmul(Q, K.transpose(-2, -1))
21     scores = scores / math.sqrt(self.embed_dim) # Scale!
22
23     if mask is not None: # For causal/decoder attention
24         scores = scores.masked_fill(mask == 0, -1e9)
```

Winter 2026

19

...continued...

Implementing self-attention in PyTorch

```
25
26     attn_weights = F.softmax(scores, dim=-1) # Normalize
27     output = torch.matmul(attn_weights, V) # Weighted sum
28
29     return output, attn_weights
30
31 # Usage example:
32 attn = SelfAttention(embed_dim=64)
33 x = torch.randn(1, 5, 64) # 5 tokens, 64-dim embeddings
34 out, weights = attn(x)
35 # out: [1, 5, 64] - contextualized embeddings
36 # weights: [1, 5, 5] - attention matrix
```


Computational complexity

Understanding the cost of self-attention

Component	Time Complexity	Memory
Self-Attention	$O(n^2 * d)$	$O(n^2)$
Feed-Forward	$O(n * d^2)$	$O(d)$

where n = sequence length, d = embedding dimension

2. Query, Key, Value Framework:

- Why three separate projections instead of one?
- What if we used $Q = K = V = X$?
- How does this relate to information retrieval?

3. Multi-Head Attention:

- Why not just use one big attention head?
- How many heads is optimal?

- Self-attention mechanism (Q, K, V)
- Multi-head attention
- Three types of attention (self, masked, cross)

Next lecture (Lecture 17 - Training Transformers):

- Positional encodings: How to inject position information
- Feed-forward networks: The other key component
- Layer normalization: Training stability
- Optimization: Making transformers faster

2. Self-Attention Mechanism

- Query, Key, Value framework
- Each token attends to all others
- Scaled dot-product: $(QK^T / \sqrt{d_k})V$

3. Multi-Head Attention

- Multiple heads learn diverse relationships
- Concatenate and project back
- Richer representations

- **Vaswani et al. (2017)** - "Attention Is All You Need"
 - The original Transformer paper
 - Introduced self-attention, multi-head attention
 - Foundation of modern NLP
- **Bahdanau et al. (2015)** - "Neural Machine Translation by Jointly Learning to Align and Translate"
 - Original attention mechanism (for comparison)

Tutorials and Resources:

- **The Illustrated Transformer** by Jay Alammar
 - <https://jalammar.github.io/illustrated-transformer/>
 - Visual step-by-step explanation
- **Annotated Transformer** by Harvard NLP
 - <https://nlp.seas.harvard.edu/annotated-transformer/>
 - Line-by-line implementation
- **HuggingFace Course** - Chapter 3

PSYC 51.17: Models of Language and Communication

Winter 2026

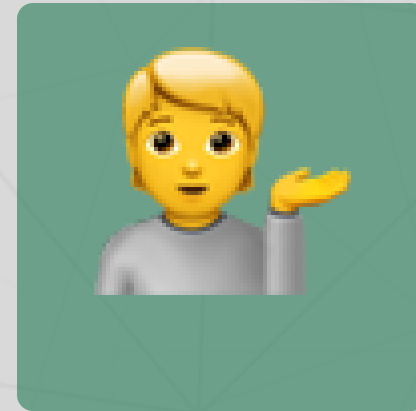
Questions?



Email me



Join our Discord



Come to office hours

Next lecture: Training transformers

Winter 2026